Entradas, lógica y tiempo real.

Hardware y firmware del entrenador

ATmega328P · DIP-28

EXPOSICIÓN FINAL · OCTUBRE 2026

ENTRENADOR AUTÓNOMO



ENTRADAS	SALIDAS
ADC + DIP	LEDs + displays

El hardware que controla el programa

ENTRADAS

Leer

Potenciómetro → ADC0.

DIP de 4 posiciones → PC1...PC4.

EJECUCIÓN

Procesar

ATmega328P a 16 MHz.

Alimentación de 5 V, RESET e ICSP.

SALIDAS

Mostrar

8 LEDs y 2 displays.

Bus compartido con 3 selecciones.

> Hardware terminado: las entradas y salidas se coordinan mediante **un firmware con cuatro modos**.

Energía estable, reloj y reinicio

9–12 V CC (Entrada) → **1N4007** (Protección de polaridad) → **LM7805** (Regulación lineal) → **5 V** (Micro y periféricos)

Filtrado y desacoplo

Entrada: 100 µF + 330 nF.
Salida: 100 µF + 100 nF.
100 nF junto a VCC y AVCC.

Reloj de 16 MHz

Cristal externo y dos capacitores de 22 pF.

La frecuencia coincide con la configuración del firmware.

RESET

Pull-up de 10 kΩ y pulsador a GND.

Permite reiniciar el programa sin desconectar la fuente.

> Alimentación y reloj estables permiten que el ADC, la temporización y el multiplexado funcionen correctamente.

El circuito prepara las señales para el firmware

Entradas

Potenciómetro de 10 kΩ

Entre 5 V y GND; cursor a PC0 / ADC0.

DIP de 4 posiciones

PC1…PC4 con pull-up internas.

Cerrado: nivel eléctrico bajo.

Salidas compartidas

8 LEDs + 2 displays 5161AS

Bus de 8 bits con resistencias de 220 Ω.

3 transistores NPN

Un grupo por transistor; base con 1 kΩ.

El firmware activa solo uno a la vez.

> Programación por ICSP: RESET · MOSI · MISO · SCK · alimentación y GND común.

Tres responsabilidades bien separadas

setup()

Configurar una vez

Dirección de los puertos.

Pull-up de las entradas.

Timer2 e interrupciones.

loop()

Calcular la respuesta

Leer potenciómetro y DIP.

Elegir el modo.

Preparar los patrones.

ISR

Refrescar cada 2 ms

Leer el patrón preparado.

Actualizar el bus.

Seleccionar un grupo.

> `loop()` escribe `pantalla[3]`; la interrupción utiliza ese búfer para actualizar las salidas.

setup(): configurar los puertos con máscaras

```
DDRD |= 0xFC;
DDRB |= 0x1F;
DDRC &= ~0x1E;
PORTC |= 0x1E;
```

Qué cambia cada instrucción

`DDRD` : PD2…PD7 como salidas.

`DDRB` : PB0…PB4 como salidas.

`DDRC` : PC1…PC4 como entradas.

`PORTC` : activa sus pull-up.

Operadores de bits

`|=` activa bits seleccionados.

`&= ~` los borra conservando los demás.

DDR define entrada/salida; **PORT** escribe salidas o habilita pull-up; **PIN** lee niveles.

De 0…1023 a una lectura estable

```
uint16_t suma = 0;
for (uint8_t i = 0; i < 8; i++)
  suma += analogRead(A0);
uint16_t nuevo = suma / 8;
```

1. Promediar

El ADC convierte la entrada analógica en un valor de 10 bits. Se promedian **8 muestras**.

2. Aplicar histéresis

Actualizar si cambia más de 2 cuentas respecto al valor estable, o si llega a 0 / 1023.

3. Escalar

`(uint32_t)pot * 100 / 1024`
Produce el número **00…99**.

> La barra usa un nivel de 0…8: `(1u << nivel) - 1` enciende los LEDs inferiores.

El DIP selecciona el modo con dos bits

```
return (uint8_t)(~PINC >> 1) & 0x0F;
```

~PINC

Invertir

Con pull-up, cerrado = 0 eléctrico. Tras invertir, cerrado = 1 lógico.

>> 1

Alinear

PC1…PC4 se desplazan a las posiciones 0…3 del resultado.

& 0x0F

Filtrar

Conservar solo los cuatro bits de los interruptores.

> `switch (dip & 0x03)` utiliza SW2/SW1 para elegir uno de cuatro modos.

Cuatro modos, una misma estructura

SW2 SW1	Rama del switch	Respuesta del firmware
00	ADC	Escala el potenciómetro a 00…99 y genera una barra de LEDs.
01	Contador	Incrementa con módulo 100; muestra decimal y binario.
10	DIP	Muestra el valor del DIP y repite sus cuatro bits en los LEDs.
11	Secuencia	Desplaza un bit y cambia de sentido en los extremos.

> Las variables `static` conservan `cuenta`, `posicion`, `sentido` y `ultimo` entre llamadas a `loop()`.

Avanzar con millis(), sin esperar con delay()

```
uint16_t periodo = 50 + pot;
bool toca = millis() - ultimo >= periodo;
if (toca) ultimo = millis();
```

Velocidad ajustable

El potenciómetro cambia el periodo entre **50 y 1073 ms**.

Actualizar cuando corresponde

`toca` habilita un paso del contador o de la secuencia.

Seguir atendiendo entradas

La aplicación no introduce una espera con `delay()` entre pasos.

> Dos escalas de tiempo: **millis()** gobierna la animación; **Timer2** mantiene el refresco de los displays.

Timer2 genera una interrupción cada 2 ms

```
cli();
TCCR2A = (1 << WGM21);
TCCR2B = (1 << CS22) | (1 << CS21);
OCR2A = 124;
TIMSK2 = (1 << OCIE2A);
sei();
```

CTC: comparación y reinicio

El temporizador cuenta de 0 a 124: **125 estados**.

Prescaler: 256

16 MHz / 256 / 125 = 500 Hz

Una interrupción cada 2 ms.

Tres grupos

El ciclo completo dura 6 ms:

≈167 Hz de refresco por grupo.

La ISR actualiza un grupo y regresa

```
ISR(TIMER2_COMPA_vect) {
  static uint8_t grupo = 0;
  PORTB &= ~0x1C;
  grupo = (grupo + 1) % 3;
  escribirBus(pantalla[grupo]);
  PORTB |= (uint8_t)(1 << (PB2 + grupo));
}
```

1 Apagar selecciones

2 Escribir el patrón

3 Activar un grupo

`volatile uint8_t pantalla[3]` : búfer compartido entre el programa principal y la ISR.

Del número al patrón de segmentos

```
pantalla[1] = reordenar(DIGITO[(n / 10) % 10], BIT_SEG);
pantalla[2] = reordenar(DIGITO[n % 10], BIT_SEG);
```

42 → 4 / 2

Separar dígitos

División entera para decenas.
Módulo para unidades.

DIGITO[]

Buscar segmentos

Tabla de patrones de 0 a 9.
Ejemplo: 0 → `0x3F`.

reordenar()

Adaptar el bus

`BIT_SEG` y `BIT_LED` relacionan bits lógicos y físicos.

Del archivo .ino al programa en Flash

FUENTE

C++ / .ino

Lógica y configuración

COMPILACIÓN

Código AVR

Binario para el ATmega328P

PROGRAMACIÓN

Flash

Arduino Uno como ISP

```
arduino-cli compile -b arduino:avr:uno firmware/entrenador
arduino-cli upload -b arduino:avr:uno -P arduinoasisp \
  -p <puerto-del-uno> firmware/entrenador
```

Antes: cargar `ArduinoISP` en el Uno y comprobar comunicación con el ATmega. Firma esperada: **1E 95 0F**.

Firmware, bootloader y fusibles

Firmware

La aplicación: entradas, modos, patrones e interrupciones.

Se almacena en Flash.

Bootloader

Programa de arranque para cargar por serial.

ISP puede escribir directamente la aplicación.

Fusibles

Configuración persistente de reloj, arranque y brown-out.

No forman parte de `loop()`.

> Configuración recomendada del proyecto: **L 0xF7 · H 0xDE · E 0xFC**.

Reloj externo full swing · arranque para bootloader · brown-out de 4,3 V.

Explicar el código mientras se observa su efecto

01 ADC · 00

Girar el potenciómetro: escalado a 00…99 y barra de LEDs.

02 Contador · 01

Cambiar el periodo y observar el paso de 99 a 00.

03 DIP · 10

Variar SW3/SW4: 02 → 06 → 10 → 14.

04 Secuencia · 11

Observar extremos, cambio de sentido y reinicio con RESET.

> Pruebas previas del repositorio: `prueba-blink` → `prueba-displays` → `entrenador`.

El firmware coordina datos, estados y tiempo

Entradas

Promedio, histéresis y máscaras convierten señales en datos útiles.

Estados

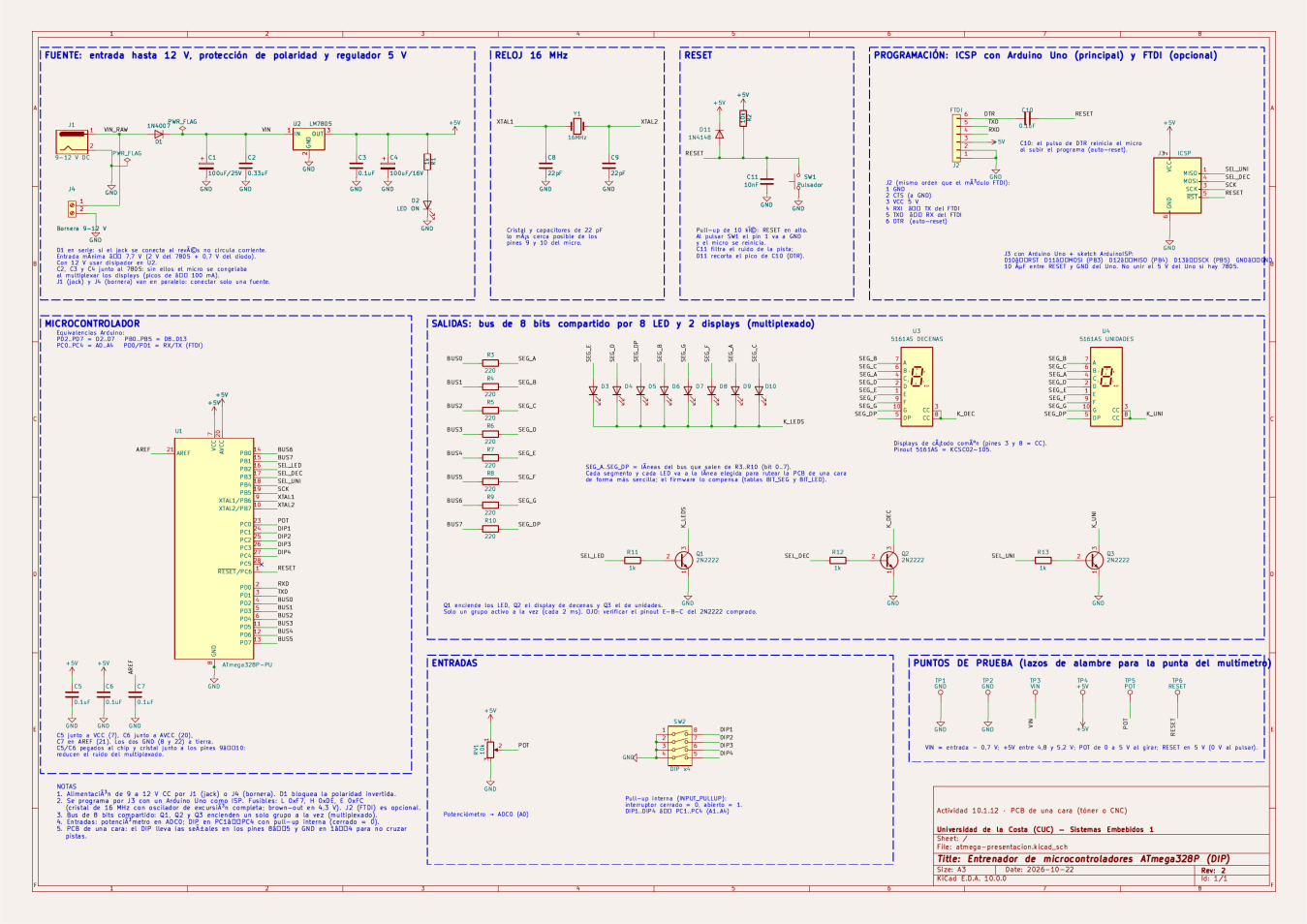
Un `switch` y variables persistentes organizan cuatro comportamientos.

Tiempo

`millis()` controla los avances; Timer2 mantiene el multiplexado.

Una aplicación en C++ · cuatro modos · ejecución autónoma

Esquemático de referencia



ANEXO B / MAPA DEL CÓDIGO

Dónde encontrar cada responsabilidad

Función / elemento	Responsabilidad
setup()	Puertos, pull-up y Timer2.
leerPot() / leerDip()	Adquisición, filtro analógico y lectura de bits.
loop()	Selección de modo, estado y control del periodo.
mostrarNumero() / mostrarLeds()	Construcción de los patrones en pantalla[].
reordenar() / escribirBus()	Traducción y escritura de los bits físicos.
ISR(TIMER2_COMPA_vect)	Refresco periódico de un grupo.

Fuente: `firmware/entrenador/entrenador.ino` · documentación: `README.pdf`

Enunciado: material del curso, sección 10.1.12